

R1 · "ESTE PROGRAMA GERA ERRO..."

"Em qual linha, por que, e como consertar?"

```
# 0 erro está na linha da COMPARAÇÃO/CONTA
# (não na linha do input!). Resposta-modelo:
# "0 erro é gerado na linha N, onde se
# compara/calcula com a variável vinda de
# input(). input() sempre retorna texto
(str),
# e comparar/operar str com int gera
TypeError.
# Conserto: converter na entrada:
# x = int(input(...)) (ou float se
decimal)"
```

R2 · "FOR OU WHILE? JUSTIFIQUE"

Dois cenários: nº fixo vs depende de condição

```
# "Caso A: for, porque o número de
repetições
# é conhecido ANTES do laço começar (N
vezes).
# Caso B: while, porque a repetição depende
de
# uma condição reavaliada a cada volta
(enquanto
# saldo/estoque permitir); o total de vezes
não
# é fixado de antemão."
# Isca: "R$40 dá 10" parece contagem, mas
saldo
# é variável → condição → while.
```

R3 · "ASSINALE AS CORRETAS (.loc)"

Trocar/ajustar valores filtrados, 5 alternativas

```
# CERTAS (2 formas equivalentes):
df.loc[df["Ano"]==2026, "Valor"] *= 0.9
df.loc[df["Ano"]==2026, "Valor"] = (
    df.loc[df["Ano"]==2026, "Valor"] * 0.9)
# ERRADAS e por quê:
# df[mask]["Valor"] = ... → cópia (chained)
# df[mask, "Valor"] → sem .loc,
inválido
# "Ano"==2026 → literal, vale
False
# df[df["Ano"]...] → sem aspas:
NameError
# "2026" vs 2026 → texto ≠ número
```

R4 · "O QUE ESTE PROGRAMA FAZ?"

Ler código pandas (3 blocos = 3x0,33)

```
# Responde em 3 LINHAS, uma por bloco:
# "1) Importa o pandas e carrega a planilha
X
# num dataframe.
# 2) Nas linhas onde [coluna]==[valor],
troca/
# multiplica [coluna-alvo] por [fator] –
preço
# passa a ser N% do original (= reduz 100–
N%).
# 3) Imprime as 5 (ou 10) primeiras linhas
do
# dataframe INTEIRO (todas as colunas)."
# *0.85 = reduz 15% · *0.2 = vira 20% (reduz
80%)
# len(série filtrada) = QUANTOS, não quais
```

R5 · INPUT + VALIDAÇÃO + FAIXAS

"Valide (int + categoria); erro → ERRO DE ENTRADA; senão calcule por faixa, adicional condicional"

```
x = input("Valor: ")
cat = input("Categoria: ").upper()
if not x.isdigit() or cat not in
["A", "B", "C"]:
    print("ERRO DE ENTRADA")
else:
    x = int(x)
    if x <= 10000:          # "até X" inclui X
        v = (x/1000) * 0.98
    elif x <= 50000:
        v = (x/1000) * 0.74
    elif x <= 100000:
        v = (x/1000) * 0.42
    else:
        v = 100           # fixo
    if cat in ["B", "C"]:
        v *= 2            # dobro vale p/
fixo tb
    print(f"R$ {v:.2f}")
```

R6 · DESCONTO ANINHADO + 3 SAÍDAS

"Desconto por ESTADO e por FAIXA do valor; apresente original, final e diferença"

```
uf = input("Estado: ")
q = int(input("Quantidade: "))
p = float(input("Preço unitário: ")) #
FLOAT!
valor = q * p
if uf == "SP":
    pct = 0.09 if valor < 3000 else 0.065
elif uf == "RJ":
    pct = 0.10 if valor < 2600 else 0.08
else:
    if valor >= 1000 and valor <= 1900:
        pct = 0.08
    else:
        pct = 0.06
final = valor * (1 - pct)
print(f"Original: R$ {valor:.2f}")
print(f"Com desconto: R$ {final:.2f}")
print(f"Diferença: R$ {valor - final:.2f}")
# pediu 3 valores → imprime os TRÊS
```

R7 · JOGO COM SALDO (while)

"Repete enquanto tiver dinheiro/estoque"

```
saldo = int(input("Saldo: "))
tentativas = 0
while saldo >= 4:          # enquanto dá pra
pagar
    saldo -= 4             # paga
    tentativas += 1
print(f"{tentativas} tentativas, sobrou
{saldo}")
```

R8 · LISTA DE STRINGS → IMPRIMIR + TOTAL

"Mostre cada item no formato X e o total" (Q5 clássica)

```
cont = 0
total = 0
for item in lista:
    partes = item.split(",") # ou "|",
"#", ";"
    a = partes[0]
    b = partes[1]
    v = float(partes[2])     # int se
inteiro
    cont += 1
    total += v
    print(f"{a} – {b} – {v} unidades")
print()                    # linha em
branco
print(f"{cont} pedidos – {total:.2f}
unidades")
# total inteiro? usa int() e imprime sem
:.2f
```

R9 · DATA JUNTA → dd/mm/aaaa

"20251119 dentro da string → 19/11/2025"

```
data = partes[2]          # "20251119"
ano = data[0:4]
mes = data[4:6]
dia = data[6:8]
print(f"{dia}/{mes}/{ano} – {valor}")
```

R10 · DATA POR EXTENSO (c/ hora opcional)

"DD/MM/YYYY [HH:MM] → 2 de outubro de 2025 às 10 horas"

```
# esqueleto completo na FOLHA 1, P8.
# Núcleo: partes = reg.split(" ") →
# len(partes)==2 detecta hora;
# meses[int(mes)-1]; singular/plural:
# h==1 "à 1 hora" / f"às {h} horas"
# m==0 omite / m==1 "e 1 minuto" / f"e {m}
minutos"
# YYYY-MM-DD? inverte: ano,mes,dia =
.split("-")
```

R11 · CONTAR/MAIOR DENTRO DO LAÇO

"Quantos itens atendem X" / "qual o maior valor"

```
cont = 0
maior = 0
for item in lista:
    v = float(item.split(",")[2])
    if v > 1000:          # a condição DELE
        cont += 1
    if v > maior:          # rastreia o
máximo
        maior = v
print(f"{cont} acima de 1000; maior:
{maior}")
```

⚠ Em TODA Q5: funciona pra QUALQUER lista (nada de fixar 4); converter o numérico do split; totalizadora FORA do laço; conferir formato caractere a caractere (espaços, hífen, linha em branco).

E1 · EXPLORAR / CONTAR	
"quantas linhas e colunas"	df.shape
"liste as colunas"	list(df.columns)
"quantos registros de cada tipo"	df["Produto"].value_counts()
"quantos tipos diferentes"	df["Produto"].nunique()
"quais são os valores"	df["Produto"].unique()
"mostre as 5 primeiras"	df.head() (10: head(10))

E2 · FILTRAR	
"só os de SP"	df[df["Estado"] == "SP"]
"acima de 5000"	df[df["Total"] >= 5000]
"SP ou RJ"	df[df["Estado"].isin(["SP", "RJ"])]
"SP E acima de 5000"	df[(df["Estado"]=="SP") & (df["Total"]>5000)]
"quantos atendem"	(df["Total"] > 5000).sum() ou len(df[mask])
"qual % atende"	(mask.sum() / len(df)) * 100

E3 · RESUMIR UMA COLUNA	
"some / média / maior / menor"	df["Total"].sum().mean().max().min()
"mín, média e máx em UMA operação"	df["Total"].agg(["min", "mean", "max"]).round(2)
"resumo completo"	df["Total"].describe()
"média só dos de SP"	df.loc[df["UF"]=="SP", "Total"].mean()

E4 · BUSCAR UM REGISTRO (id)	
pid = int(input("id: ")) # tipo = o da coluna! if pid not in df["id"].values: print("Não encontrado") else: linha = df[df["id"]==pid].iloc[0] print(f"... {linha['nome']} ... {linha['idade']}") # árvore de decisão completa: FOLHA 1, D6	

E5 · ALTERAR / CRIAR COLUNA	
"troque o valor dos selecionados"	df.loc[df["Cod"]==cod, "Preço"] = novo (variáveis do input!)
"reajuste de +0,04 só de dezembro"	df.loc[df["Mês"]==mes, "Preço"] += reajuste
"desconto de 10% nos de SP"	df.loc[df["UF"]=="SP", "Preço"] *= 0.9
"crie coluna com o total"	df["Total"] = df["Preço"] * df["Qtd"]
"calcule o preço unitário"	df["Preço Unit"] = df["Total"] / df["Qtd"]
"classifique lucro/prejuízo"	(df["M"]<0).replace({True:"prejuízo", False:"lucro"})
"coluna mês da data e texto"	df["Data"].str[6:10] + "-" + df["Data"].str[3:5] (DD/MM/AAAA→AAAA-MM)
"prefixo do código"	df["Cod"].str[0:3]

E6 · FATURAMENTO ANTES x DEPOIS	
"diferença com e sem o reajuste" total_pre = df["Total"].sum() # guarda ANTES df["Preço Unit"] = df["Total"]/df["Qtd"] df.loc[df["Mês"]==mes, "Preço Unit"] += reajuste df["Total"] = df["Preço Unit"] * df["Qtd"] dif = df["Total"].sum() - total_pre print(f"Diferença: R\$ {dif:.2f}") # reajuste/aumento = + desconto = -	

E7 · "MÉDIA/TOTAL POR GRUPO" (groupby)	
"média de preço por estado"	df.groupby("Estado")["Preço"].mean()
"...ordenado / ranking"	...sort_values(ascending=False)
"QUAL estado tem a maior média"	df.groupby("Estado")["Preço"].mean().idxmax()
"por estado E produto"	df.groupby(["Estado", "Produto"])["Preço"].mean().reset_index()
"vários stats por grupo"	df.groupby("Estado")["Preço"].agg(["mean", "max"])
"top 5 / 5 menores"	serie.sort_values(ascending=False).head(5) · menores: sort_values().head(5)

E8 · "GRADE DE X POR Y" (pivot_table)	
"preço médio por município e mês, um produto por coluna" tab = df.pivot_table(index=["Município", "Mes"], # Linhas columns="Produto", # vira colunas values="Valor", # o que resumir aggfunc="mean").reset_index() tab.columns.name = None # 1 chave só no index? sem a lista: index="UF"	

E9 · "CRUZE AS DUAS TABELAS" (merge)	
dados = pd.merge(pedidos, custos, on=["Código", "Mês"]) # TODAS # as chaves em comum! # teste: dados.shape preserva as linhas? # depois: dados["Margem"] = (dados["Venda"] - dados["Custo"])	

E10 · CARREGAR (antes de tudo)	
Excel	pd.read_excel("a.xlsx") (+ index_col=0 se 1ª col = índice)
CSV brasileiro	pd.read_csv("a.csv", sep=";", decimal=",")
conferir	df.dtypes → valor tem que ser float64/int64

E11 · QUESTÃO INTERPRETATIVA (modelo)	
"o líder do ranking é confiável? / dá pra decidir só com isso?" # Molde de resposta (adapta os colchetes): # "Pela letra do enunciado, a resposta é [X]. # Porém [X] tem apenas [N] registros (ver # value_counts): amostra pequena, média frágil # a outliers. Entre os grupos com volume # relevante, [Y] lidera. # Além disso, os dados NÃO capturam: custos [da franquia/logísticos], impostos, volume # real, concorrência local e risco. A decisão # exige comparar a métrica LÍQUIDA desses # fatores, não a margem bruta isolada."	

⚠ Roteiro: 1) acha o VERBO do pedido → seção desta folha; 2) copia o molde; 3) troca coluna/valores pelos do enunciado (df.columns!); 4) roda; 5) confere com head()/shape antes de seguir.